

# Introduction to programming a robot

University of Adelaide  
David Knight

Processed with L<sup>A</sup>T<sub>E</sub>X on January 12, 2015

## 1 What is programming?

We tend to think of programming as being very “20th-century”, but the truth is that humans have been programming for many centuries. Consider some examples:

A skilled and artistic wool-worker can create a wool pullover by choosing wool and knitting-needles, and manipulating them in a clever way. If instead the worker writes down the pattern, many less-artistic and less-skilled workers, who understand the words of knitting: *knit*, *purl*, *drop-a-stitch*, etc, can manipulate the knitting-needles and produce an equally beautiful pullover. Indeed a roomful of such workers can produce many beautiful pullovers — far more than the skilled-worker could ever produce alone.

A skilled chef can create a delicious chocolate cake, by choosing suitable ingredients and combining and cooking them appropriately. If instead the chef prepares a *recipe*, less skilled cooks that understand basic cooking terms, such as *mix*, *fold*, *stir*, *oven*, etc will be able to reproduce the chocolate cake at any time.

A highly skilled musician is able to invent tunes and play them on a musical instrument. If the skilled musician takes the role of *composer*, and writes the tune down in musical notation, less-skilled musicians, who understand musical notation: *clef*, *sharp*, *key*, *minim*, etc, can play the tune just as well as the composer.

More recently, before the advent of modern digital computers, (prior to the second world war) complex numerical problems, such as calculating the path of a projectile fired from a gun, were solved using mechanical calculators operated by humans (mostly women, because they were more careful than men). The sequence of steps required to solve the problem was invented by skilled mathematicians, and then executed repeatedly by the human “computers” until the problem was solved. The computers understood the mathematical operations such as *add*, *multiply*, *clear*, *store*, etc. Once written down, the steps could be used to solve the problem for different gun sizes, elevation angles, quantity of explosives, and wind speed and direction.

We see that any time we write down detailed instructions describing how to solve a problem, we are “programming”, and the main reason for programming is to allow us to delegate the actual work of solving the problem to another person, or to a machine.

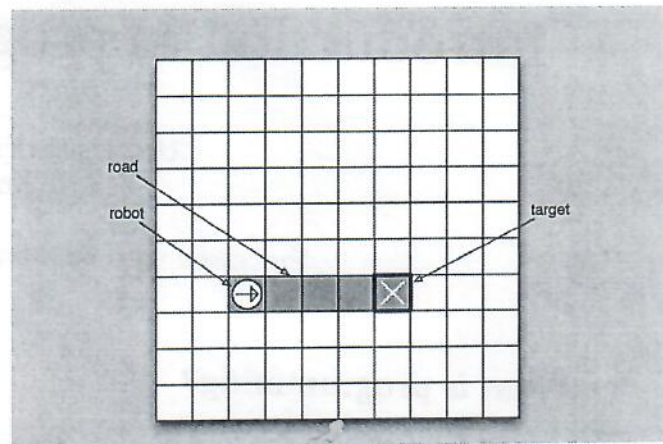
## 2 Programming is a strange business

Programming is a strange business, because at one moment, while we are inventing a way to solve the problem, we are being *clever creative humans*. But in the next moment, when we are checking to see if our program works, we play the role of the *mindless dumb machine* that will execute our program. We flip back and forth between these two mode of thinking as we work on a problem, until it is solved.

### 3 Introduction to the robot

Imagine we have a very simple robot that can move in a rectangular grid, as shown here:

The grey squares mark roads that the robot can travel along.



Here is the robot:

It is currently facing to the right, as shown by the arrowhead.



Here is the mark that shows the target that the robot seeks:



Our robot can obey two commands, that cause it to move:

- Go forward  $x$  steps; and
- Turn left  $45x$  degrees.

It also has two sensors to enable it to “see” the world:

- Check to see if the robot is *on* the target; and
- Check to see if there is a wall *directly in front* of the robot.

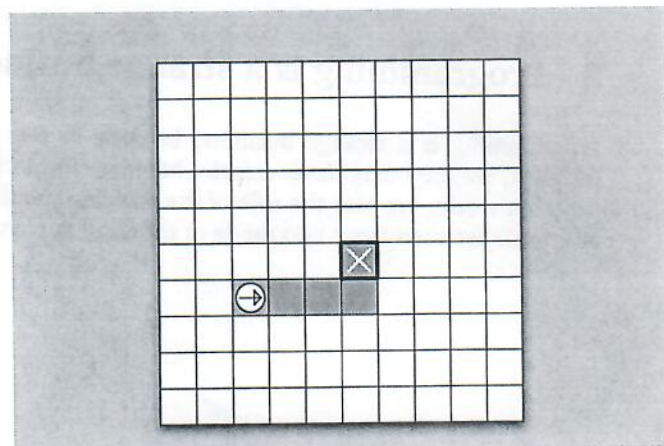
Your task is to give instructions to navigate the robot from its starting position to the target, in this map:

In this map, the solution is simply:

Go forward 3

Turn left 2

Go forward 1





## 4 The instructions that the robot's computer understands

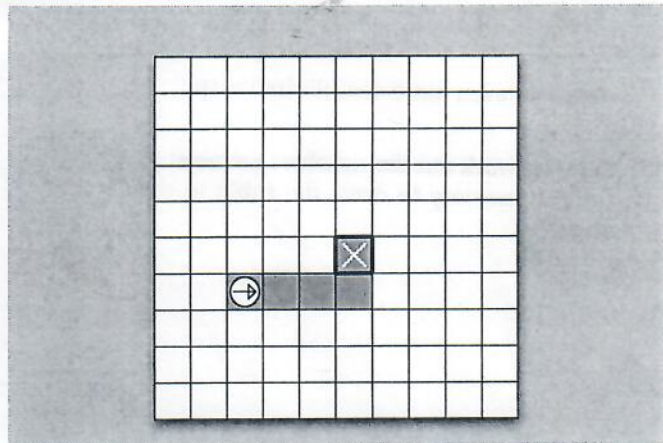
Unfortunately, our robot doesn't understand commands given in English words. It is controlled by a (very tiny) computer, that needs the instructions to be *numbers*. Each instruction is a two-digit decimal number, like this:

| Number | Mnemonic     | Meaning                                                       |
|--------|--------------|---------------------------------------------------------------|
| 00     | hlt          | halt (stop executing instructions)                            |
| 1x     | fwd <i>x</i> | Move the robot forward <i>x</i> squares                       |
| 2x     | trn <i>x</i> | Turn the robot left $45x$ degrees                             |
| 3x     | jmp <i>x</i> | Jump to location <i>x</i>                                     |
| 4x     | jit <i>x</i> | Jump to location <i>x</i> , if the robot is on the target     |
| 5x     | jnt <i>x</i> | Jump to location <i>x</i> , if the robot is not on the target |
| 6x     | jiw <i>x</i> | Jump to location <i>x</i> , if the robot is facing a wall     |
| 7x     | jnw <i>x</i> | Jump to location <i>x</i> , if the robot is not facing a wall |
| 8x     | inc <i>x</i> | Add 1 to the number in location <i>x</i>                      |
| 9x     | dec <i>x</i> | Subtract 1 from the number in location <i>x</i>               |

Here is the map we saw previously:

Written as numbers our solution is now: 13  
22 11 00

Notice that at the end of the instructions we must include a 00 *hlt* instruction.



However, this is not quite like a real computer yet...

Inside our computer, the instructions are stored in a (very tiny) memory that has space for just 10 numbers, so the program really looks like this:

| Location | Contents | Meaning |
|----------|----------|---------|
| 0        | 13       | fwd 3   |
| 1        | 22       | trn 2   |
| 2        | 11       | fwd 1   |
| 3        | 00       | hlt     |
| 4        | 34       | (junk)  |
| 5        | 26       | (junk)  |
| 6        | 71       | (junk)  |
| 7        | 55       | (junk)  |
| 8        | 01       | (junk)  |
| 9        | 99       | (junk)  |

When we start the computer, it always begins by executing the instruction in location 0:

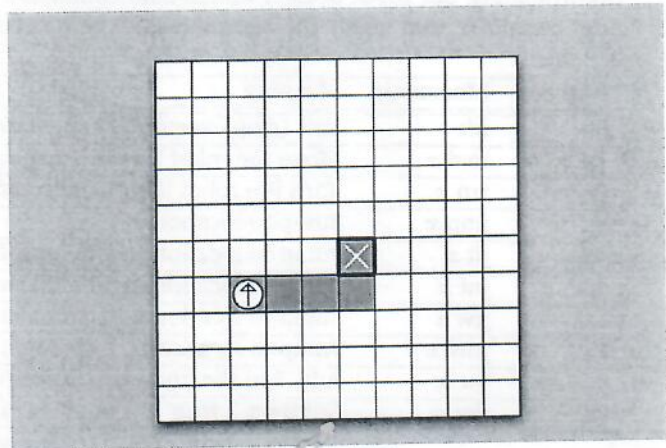
- That's the 13= *Move forward 3* instruction.
- Then it executes the instruction in location 1 — 22= *Turn left  $2 \times 45$  deg.*
- Then it executes the instruction in location 3 — 11= *move forward 1.*
- Then it executes the instruction in location 4 — 00= *hlt.*
- And it stops executing instructions.

Notice that *all* the memory locations contain numbers (just like a real computer), but we don't care what some of them are, (locations 4 onwards, marked (junk) never get executed, because of the *hlt* instruction.)

## 5 Some more complicated situations

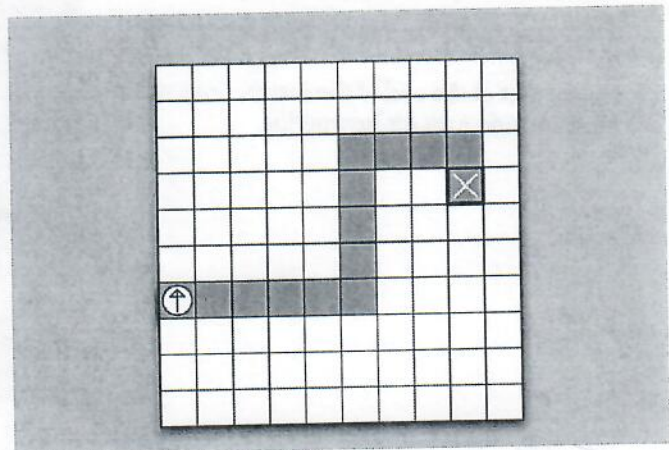
Here's a more complicated situation:

Can you work out the numbers we need to store in memory to drive the robot to the target?



Here's an even more complicated map:

Can you work out the numbers we need to store in memory to drive the robot to the target?



The solution to the last problem is so long that it nearly fills up our computer's (tiny) memory — we need to be more cunning!

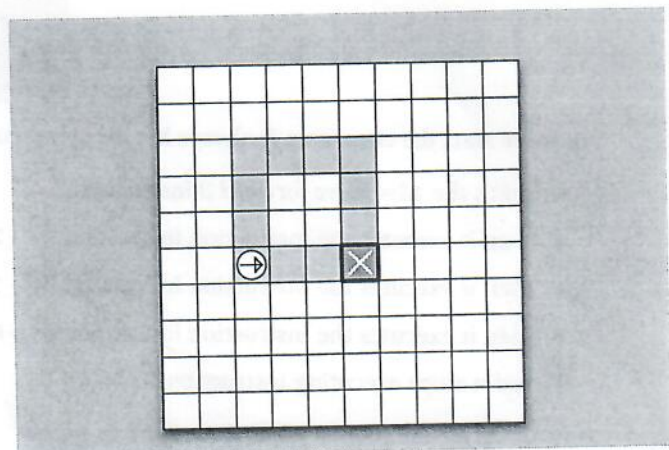
## 6 The *Jump* instruction

Our computer includes a *jump* instruction, that allows us to change the order of execution of instructions. When the computer encounters a *3x (jmp x)* instruction, it “jumps” so that the next instruction is fetched from location *x* (instead of from the next location, as usual).

Consider this map:

And this program in the computer:

| Location | Contents | Meaning |
|----------|----------|---------|
| 0        | 22       | trn 2   |
| 1        | 13       | fwd 3   |
| 2        | 24       | trn 4   |
| 3        | 30       | jmp 0   |
| 4        | 43       | (junk)  |
| 5        | 62       | (junk)  |
| 6        | 75       | (junk)  |
| 7        | 52       | (junk)  |
| 8        | 81       | (junk)  |
| 9        | 69       | (junk)  |





If we execute this program we see that the robot moves around the square of roads forever, in a *clockwise* direction, and never stops at the target. We'll fix this problem soon — stay tuned!

A program that executes the same instructions repeatedly, over and over again, is said to be *looping*. In this program, each loop of the program executes four instructions. Clearly, the time taken for the computer to execute a loop depends on the number of instruction it executes each time. Can you see a way to make the program execute only *three* instruction in the loop, instead of four, and still go around the square map?

One last variation: can you devise a program that will make the robot move around the square forever in an *anticlockwise* direction?

## 7 The target-sensor instructions

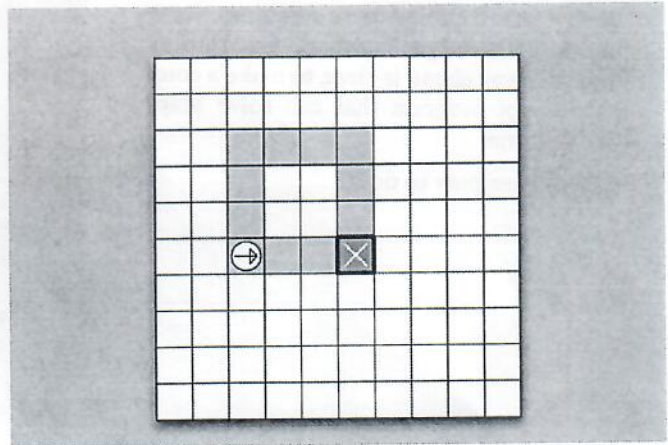
As noted earlier, our robot has some sensors — it can detect when it is on the target square. The control computer has two instructions that examine the value of each sensor:

- $4x$ , (*jit*  $x$ ), which means: Jump to location  $x$  if the robot is on the target square; and
- $5x$ , (*jnt*  $x$ ), which means: Jump to location  $x$  if the robot is *not* on the target square

Here's the map we had earlier:

And here's the new program:

| Location | Contents | Meaning |
|----------|----------|---------|
| 0        | 22       | trn 2   |
| 1        | 13       | fwd 3   |
| 2        | 26       | trn 6   |
| 3        | 51       | jnt 1   |
| 4        | 00       | hlt     |
| 5        | 26       | (junk)  |
| 6        | 71       | (junk)  |
| 7        | 55       | (junk)  |
| 8        | 01       | (junk)  |
| 9        | 99       | (junk)  |



Can you see how it works?

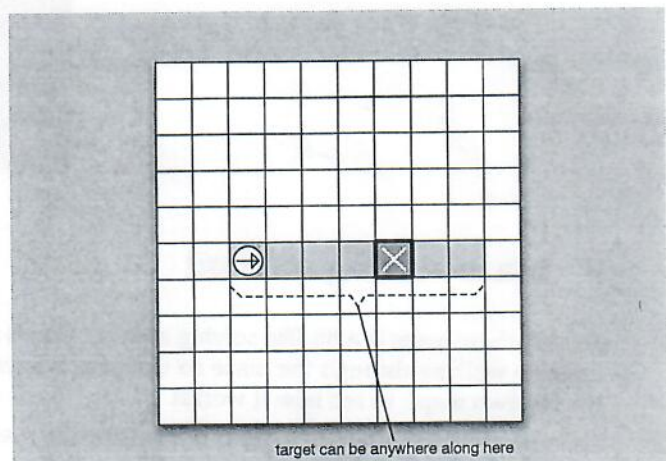
This is definitely better! It will find the target no matter which corner we put it on. But there's actually an error in this program — it can behave in a silly way. Can you find the test condition that causes the strange behaviour? Hint: Imagine what happens if the target is moved to each of the four corners.

But that's not the end of our problems. What happens if the target is in the middle of one of the sides and not at a corner? A moment's thought tells us that the robot will go right past the target. We'll solve this problem now, by starting with a simpler case.

Consider this map. Suppose the target can be located anywhere along the road.

The idea is to take *one* step, and then check to see if we have found the target — if not, we move again

Can you make a program that will drive the robot the right distance to the target, no matter where it is?





Does your program work correctly if the robot is already on the target when the program starts running? What do you have to change to make it work correctly?

## 8 The wall-sensor instructions

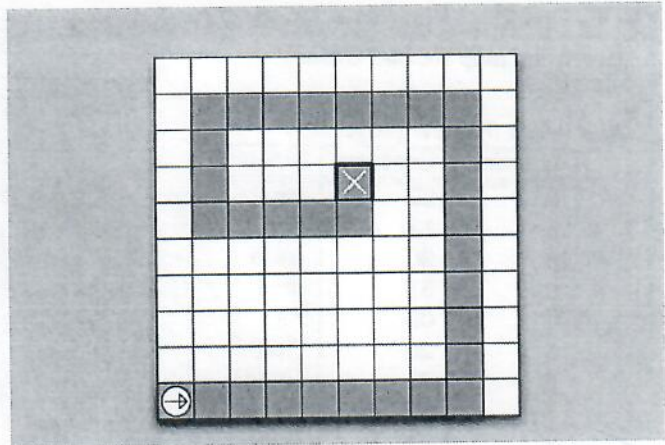
Now we need to deal with corners in the map. A corner can be detected when there's a wall directly in front of the robot. Our robot has a wall sensor, and the computer has two instructions that will allow it to test the output of the sensor, in a way that is similar to the target-sensor instructions:

- $6x, (jiw\ x)$ , which means: Jump to location  $x$  if the robot *sees* a wall immediately in front of it; and
- $7x, (jnw\ x)$ , which means: Jump to location  $x$  if the robot *does not* see a wall immediately in front of it.

We can use the wall-sensor instructions to make the robot turn whenever it sees a wall. Can you see how to use the instructions, so that the robot will keep turning until the way ahead is clear (but won't turn if it is already clear)?

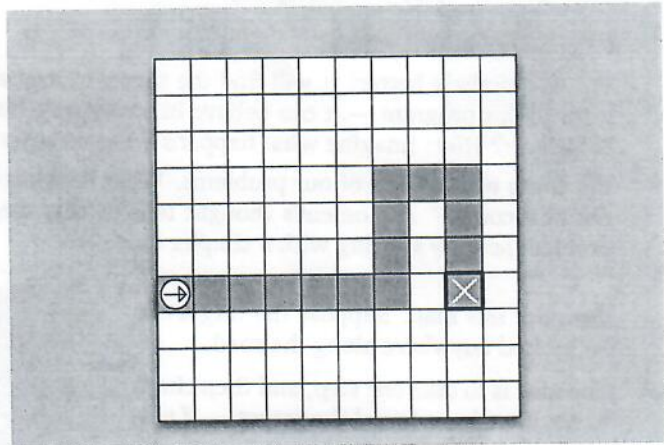
We can now combine these two ideas: moving until the target is found, and turning until the way ahead is clear, to make a complete robot program that can solve maps like this one:

Can you see how to do it?



Unfortunately, this is still not enough. What does your program do when confronted with this map?

Does it get stuck?

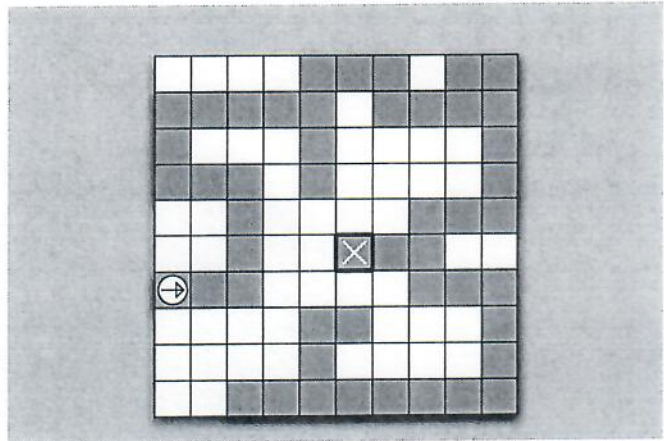


## 9 Maze-solving method

Solving these maps is a bit like solving a maze. Here's a maze solving technique that works for *some* mazes: Imagine walking through the maze so that you always have your left-hand touching the wall. Try it out on the last two maps, to see how it works!

Can you write the program to do this job using the machine instructions? Hint: How can you tell if there's a wall on your left-hand side?

Here's a really complicated map that can be solved using the maze algorithm.



## 10 Looking back

We have been solving problems for a very simple robot, but the thought-processes that underlie the programs we wrote are *exactly* the same as the ones used to build any piece of modern software. If you can solve these problems, you can be a programmer! If you found it fun, you *should* be a programmer!

In addition we explored a very simple computer, controlled entirely by numbers. This machine works *exactly* the same way as real computers, the only difference is real computers can handle much bigger programs because the memory is bigger (typically four *billion* locations, instead of just 10), and the machine has more kinds of instructions (typically about 200, instead of 10).

The people who programmed computers in 1945 wrote their programs entirely in numbers, exactly as we have done — but that's too hard for the modern age.

## 11 Looking forward

Since the mid 1950s, computer languages have been invented that allow us to express programs in a way that is convenient for humans, and can also be machine-translated into the numbers needed for the computer.

Our last program, that solves maps by keeping its left-hand on the wall can be expressed in a modern way, like this:

```
loop until robotAtTarget() do
  robotTurn(2)
  loop while robotSeesWall() do
    robotTurn(6)
  endloop
  robotForward(1)
endloop
```